The Judge's Experience: A 3-Minute "Golden Path"

This is the narrative we are building. The UI and backend must serve this story exclusively.

Scene 1: The Landing (0-30 seconds)

The judge opens the URL. There is no empty form. They are immediately presented with a live, pre-configured "Palantir-style" dashboard.

- **What they see:**
  - A dark-themed interface with electric blue and white highlights.
  - **Top Left:** The title "HedgeLogic: Certainty Engine" and the Polymarket market being tracked (e.g., "Will Fed raise rates in Q4?").
  - **Center Stage:** A large, pulsing donut chart showing **"Current Hedge: 30%"**. Inside the donut, a big number shows the **"Premium Paid: $3,000"**.
  - **Right Panel:** A live-updating line chart showing the market's price (probability) over the last hour, with the current price clearly labeled: **"Current Probability: 41¢"**.
  - **Bottom Panel:** An activity log, already populated:
    - `[10:31:00 AM] System monitoring. Price: $0.41. No action needed.`
    - `[10:30:00 AM] CRON JOB: Trigger activated (Price $0.40 > $0.20). Executing trade...`
    - `[10:30:02 AM] TRADE CONFIRMED: Bought 3,000 contracts. New hedge: 30%.`
- **The feeling:** The judge has walked into a live mission control. The system is already working, it's real, and it looks professional.

Scene 2: The Interaction (30 seconds - 2 minutes)

This is where the judge takes control. They don't fill out a boring form; they manipulate the strategy directly on the dashboard.

- **What they do:**
  1. On the left panel under "Strategy," they see a slider labeled **"Tier 1 Trigger"** currently set at **20¢**.
  2. They see a second slider, **"Tier 2 Trigger"**, currently set at **50¢**.
  3. The current price is 41¢, which is above the first trigger but below the second.
  4. The judge **drags the "Tier 2 Trigger" slider down from 50¢ to 35¢**.
  5. A "Commit Strategy Change" button glows. They click it.

- **What they see:**
    - The UI immediately responds. A loading spinner appears on the button. A new entry appears in the activity log: `[10:32:15 AM] USER ACTION: Strategy updated. New Tier 2 trigger set to $0.35.`
    - Now, they wait. The cron job runs every minute. This anticipation is part of the drama.
    - Within the next 60 seconds, the dashboard comes alive:
        - A new log entry appears: `[10:33:00 AM] CRON JOB: Trigger activated (Price $0.41 > $0.35). Executing trade to reach 80% hedge...`
        - A second later: `[10:33:02 AM] TRADE CONFIRMED: Bought 5,000 contracts.`
        - The big donut chart animates, smoothly filling up from **30% to 80%**.
        - The "Premium Paid" number ticks up to `$7,400`.
- **The feeling:** The judge just commanded the system. They made a decision, and the machine executed it, providing clear, immediate visual feedback. This is the "Aha!" moment.

Scene 3: The Proof (2 - 3 minutes)

The judge proves it's not a canned demo.

- **What they do:**
    1. They see an input box labeled "Track New Market."
    2. They go to Polymarket, find another "Yes/No" market, and paste the URL into the box.
    3. They click "Track."
- **What they see:**
    - The entire dashboard resets and reloads.
    - The title changes to the new market's name. The chart clears and starts plotting the new price. The hedge is at 0%.
    - They can now set new triggers for this new market and repeat Scene 2.
- **The feeling:** This is a real, dynamic tool, not a hardcoded video.

The 10-Hour Battle Plan: Divide and Conquer

Team Structure:

- **Dev 1 (Frontend Palantir):** Owns the UI. Lives in React. Their only goal is making the "Golden Path" experience look and feel incredible.
- **Dev 2 (Backend Worker):** Owns the Cloudflare Worker and D1. Lives in TypeScript. Their goal is to make the cron job and API endpoints work reliably.

- **Dev 3 (API & Glue):** Owns the Polymarket integration and deployment. They are the bridge between Dev 1 and Dev 2.

Phase 1: Setup & The API Contract (Hours 0-1.5)

- **ALL HANDS (30 mins):** The most critical meeting. Agree on the **exact JSON structure** for the two API endpoints. Write it in a shared document.
  - `GET /api/status` : What fields are needed to render the entire dashboard? (`marketName`, `currentPrice`, `priceHistory`, `currentHedgePercent`, `totalPremium`, `strategyTiers`, `activityLog[]`).
  - `POST /api/strategy` : What fields does the UI send? (`marketId`, `maxExposure`, `tiers[]`).
- **Dev 3 (Glue):** Create the GitHub repo. Set up Cloudflare Pages and the Worker project. Set up D1 with the two simple tables. Deploy the "hello world" versions of both.
- **Dev 1 (Frontend):** `npx create-vite@latest --template react-ts`. Install `tailwindcss` and **Tremor**. This is key for the Palantir look. `npm install @tremor/react`.
- **Dev 2 (Backend):** `wrangler init` the Worker project.

Phase 2: Mock All The Things (Hours 1.5-4)

- **Dev 1 (Frontend):** Build the entire UI with **hardcoded data**. It should look exactly like the final product, but be completely static. Use the JSON structure from the API contract. The sliders and buttons don't have to *do* anything yet, just exist. **Goal: A beautiful, non-functional dashboard by hour 4.**
- **Dev 2 (Backend):** Build the `GET /api/status` and `POST /api/strategy` endpoints in the Worker. They should interact with the D1 database but **return hardcoded mock data**. Do not call the real Polymarket API yet. **Goal: Endpoints that Dev 1 can successfully call, even if the data is fake.**
- **Dev 3 (Glue):** Write a simple, standalone Node.js script that successfully: 1) Fetches the price for a market from Polymarket. 2) Executes a trade on Polymarket. This de-risks the external API integration completely. Document the exact `fetch` calls needed.

Phase 3: Wire the Frontend (Hours 4-7)

- **Dev 1 (Frontend):** Wire the UI to the live Worker endpoints. Replace the hardcoded data with live `fetch` calls. Implement the polling on the `GET /api/status` endpoint. Make the "Commit Strategy" button `POST` the data from the sliders. **Goal: A UI that displays data from the worker and can successfully update a strategy in D1.**
- **Dev 2 & Dev 3 (Backend Pair-Programming):** Now, make the backend real.

    - In the Cron job handler, replace the mock price with a real `fetch` call to Polymarket (using the code Dev 3 already wrote).
    - Implement the core decision logic: `if (price > trigger) then calculate_trade()`.
    - Implement the trade execution `fetch` call.
    - Make sure every step writes to an `ActivityLog` table in D1.
    - Update the `GET /api/status` endpoint to pull real data from D1 and Polymarket. **Goal: A fully functional cron job that executes real trades based on a strategy set by the UI.**

Phase 4: E2E Testing & Polish (Hours 7-10)

- **ALL HANDS:** Test the "Golden Path" from end to end. Find all the bugs.
- **Dev 1 (Frontend):** Add the final polish. Loading states, error messages, smooth animations on the charts. Tweak CSS to perfection.
- **Dev 2 (Backend):** Watch the logs with `wrangler tail`. Harden the logic. Make sure it's robust.
- **Dev 3 (Glue):** Prepare the demo. Pre-configure a strategy in the database so the judge's first view is the impressive "Scene 1". Write the 30-second pitch.

This plan is aggressive and leaves no room for error, but it is achievable. The key is strict role definition and a non-negotiable, pre-defined API contract between the frontend and backend.